CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati
saradhi@iitg.ac.in

Week #06 Lab Class Inheritance & Matrix Determinant

Week06 Lab Class Inheritance & Matrix Determinant

```
class matrix
{
  private:
    int rows, columns;
    float data[3][3];
  public:
    matrix()
    {
      //Initialize matrix
    }
    void lu()
    {
      cout << "LU Decomposition\n";
    }
};
```

Week06 Lab Class Inheritance & Matrix Determinant

```
class matrix
{
  private:
    int rows, columns;
    float data[3][3];
  public:
    matrix()
    {
      //Initialize matrix
    }
    void lu()
    {
      cout << "LU Decomposition\n";
    }
};
```

```
Task 1.1 - No issue

Task 1.2a -

Matrix(int r = 4, int c = 4)
{
  // Initialize appropriately
}

Task 1.2b -
Matrix(float **f, int r, int
{
  // Initialize appropriately
}
```

Week06 Lab Class Inheritance & Matrix Determinant

```
Task 1.2c -

matrix(const matrix &m)
{
   //Initialize matrix
}

Task 1.2d -

pair<matrix, matrix>lu()
{
  // Do the work
  Return make_pair(L, U);
}
```

Week06 Lab Class Inheritance & Matrix Determinant

```
Task 1.2e - Determinant

float det(const Matrix L, const Matrix U);

Task 1.2f - overloaded >>
No issue

Task 1.2g - overloaded <<
No issue

Task 1.2h - Destructor
No issue

Task 1.2i - overloaded *
No issue
```

Week06 Lab Class Inheritance & Matrix Determinant

```
Task 1.2j - overloaded -
No issue

Task 2.1 -
No issue

Task 2.2 -
class Identity_Matrix : public matrix
{
using matrix::matrix;
   Identity_Matrix() : matrix();
}

Task 2.3 - Copy constructor
No issue
```

Week06 Lab Class Inheritance & Matrix Determinant

```
Task 2.4 -
void det()
{
    // Print |I|
}

Task 2.5 -
Here is the important thing:
void lu()
{
  matrix::lu();
}

Task 2.6 -
No issue
```

Week06 Lab Class Inheritance & Matrix Determinant

```
Task 2.7 -
No issue

Task 2.8 -
No issue

Task 2.9 -
No issue

Task 2.10 -
No issue

Task 2.11 -
No isse
```

Week06 Lab Class Inheritance & Matrix Determinant

```
Task 3 - Similar to Task 2

Task 4 - Similar to Task 2

Task 5 -
No issue
```

Week06 Lab Class Inheritance & Matrix Determinant

```cpp
#include<iostream>
using namespace std;

class matrix
{
  private:
    int r, c;
    int m[3][3];

  public:
    matrix(int r= 3, int c = 3)
    {
      cout << "Default" << endl;
    }
    void lu()
    {
      cout << "LU Decomposition" << endl;
    }
};
```

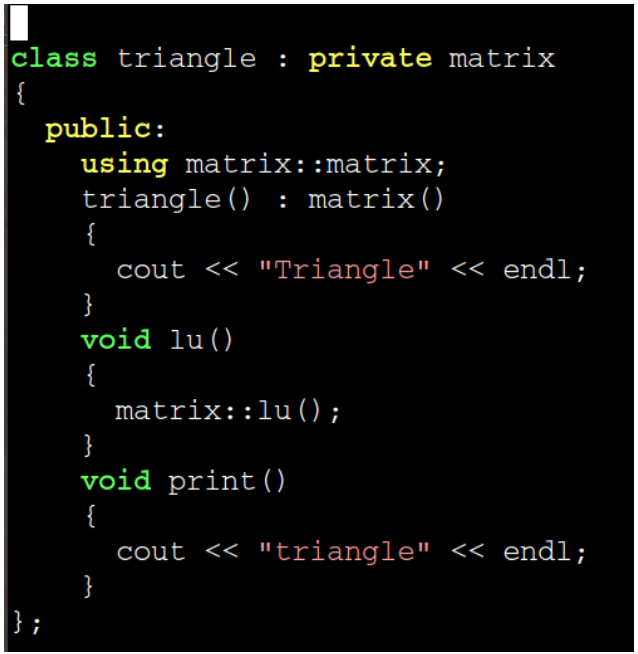
```cpp
class triangle : public matrix
{
  public:
    using matrix::matrix;
    triangle() : matrix()
    {
      cout << "Triangle" << endl;
    }
    void lu()
    {
      matrix::lu();
    }
    void print()
    {
      cout << "triangle" << endl;
    }
};
```

Week06 Lab Class Inheritance & Matrix Determinant

```
int main(int argc, char *argv[])
{
  matrix m;
  m.lu();

  // Solution #1
  triangle t;
  t.lu();

  // Solution #2 cannot be performed due to access level
  //t.lu();
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ g++ week06.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ ./a.out
Default
LU Decomposition
Default
Triangle
LU Decomposition
```

Week06 Lab Class Inheritance & Matrix Determinant



```
class triangle : private matrix
{
  public:
    using matrix::matrix;
    triangle() : matrix()
    {
      cout << "Triangle" << endl;
    }
    void lu()
    {
      matrix::lu();
    }
    void print()
    {
      cout << "triangle" << endl;
    }
};
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ g++ week06-private.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ ./a.out
Default
LU Decomposition
Default
Triangle
LU Decomposition
```

Week06 Lab Class Inheritance & Matrix Determinant

```
class triangle : protected matrix
{
  public:
    using matrix::matrix;
    triangle() : matrix()
    {
      cout << "Triangle" << endl;
    }
    void lu()
    {
      matrix::lu();
    }
    void print()
    {
      cout << "triangle" << endl;
    }
};
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ g++ week06-protected.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ ./a.out
Default
LU Decomposition
Default
Triangle
LU Decomposition
```

Week06 Lab Summary

- Many students' claims error in question.
- These claims are incorrect.
- The following are the reasons for incorrectness in students' claim.
- In base class lu() has public access
- The lu() function can be accessed from main using base class (matrix) object

Week06 Lab Summary

- Identity_Matrix is derived in public mode from base class.
- In main, creating Identity_Matrix object, initializing and finding determinant is no issue.
- Diagonal_Matrix derived in protected mode
- Access level of base class lu() will change to protected.
- There is no issue accessing matrix::lu() from withing Diagonal_Matrix class;
- You cannot access matrix::lu() from main function

Week06 Lab Summary

- Triangular_Matrix derived in private mode
- Access level of base class lu() will change to private.
- There is no issue accessing matrix::lu() from withing Diagonal_Matrix class;
- You cannot access matrix::lu() from main function!
- Any one of the solution stated above is acceptable!

Tutorial #06 Virtual Functions & Templates

Overview

- **Virtual Functions**
- **Templates**

Virtual Functions

Virtual Functions

- A virtual function is a function that is declared within a base class and is meant to be overridden in derived classes.
- Virtual functions allow you to achieve runtime polymorphism, meaning that the decision of which function to call is made at runtime rather than at compile time.
- This is particularly useful when you want to use a base class pointer or reference to call a function that may be overridden by a derived class.
- When a function is declared as virtual in a base class, the derived class can override it, and the correct function (the derived version) will be called based on the actual object type, not the type of the pointer or reference.

Virtual Functions

- Virtual functions enable the selection of the appropriate function at runtime.
- You can use a pointer or reference to the base class to refer to objects of derived classes.
- A derived class can provide a specific implementation of a virtual function.
- A function declared with `= 0` in the base class is called a pure virtual function, making the class abstract.

Virtual Functions

```cpp
#include <iostream>
using namespace std;

class Base
{
public:
    virtual void show()
    {
        cout << "Base class show function" << endl;
    }
};

class Derived : public Base
{
public:
    void show() override
    {
        cout << "Derived class show function" << endl;
    }
};
```

Virtual Functions

```
int main(int argc, char *argv[])
{
    Base* basePtr;
    Derived derivedObj;

    basePtr = &derivedObj;

    basePtr->show();

    return 0;
}
```

Thunderbird Mail

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ g++ vf01.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ ./a.out
Derived class show function
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$
```

Pure Virtual Functions – Abstract Class

Pure Virtual Functions

- A pure virtual function is a virtual function that is declared in a base class but is not defined within that class.
- The base class expects derived classes to provide their own specific implementation of the function.
- Pure virtual functions are used to create abstract classes
- Abstract classes guide the hierarchy of classes or derived classes
- `virtual void myFunction() const = 0;`
- Any class which has this function definition is an abstract class

Pure Virtual Functions

- A class that contains at least one pure virtual function is called an abstract class.
- Abstract classes cannot be instantiated directly.
- They are designed to be base classes that provide a common interface for derived classes.

```
class A
{
  public:
    virtual void print() = 0;
};

int main(int argc, char *argv[])
{
//Object cannot be instantiated
  A a;
}
```

Pure Virtual Functions

- If derived classes inherit from abstract class they must
- Implement print() or
- Act further as abstract class

```
class A
{
  public:
    virtual void print() = 0;
};

class B : public A
{
  public:
    void print()
    {
        cout << "Hello B.print()\n";
    }
}
```

Pure Virtual Functions

- If derived classes inherit from abstract class they must
- Implement print() or
- Act further as abstract class
- That is derived classes must override pure virtual functions; otherwise, they will also become abstract classes and cannot be instantiated.

```
class A
{
  public:
    virtual void print() = 0;
};

class B : public A
{
  public:
  // Not implemented void print()
  // B becomes abstract class
}
```

Pure Virtual Functions

- `Shape` is an abstract class because it contains a pure virtual function `draw()`.
- `Circle` and `Square` are derived classes that override the `draw()` function.
- In the `main` function, `Shape*` pointers are used to call `draw()`, and the correct function is called based on the actual object type.

Virtual Functions

```cpp
#include <iostream>
using namespace std;

class Shape
{
public:
    virtual void draw() const = 0;
};

class Circle : public Shape
{
public:
    void draw() const override
    {
        cout << "Drawing a Circle" << endl;
    }
};
```

```cpp
class Square : public Shape
{
public:
    void draw() const override
    {
        cout << "Drawing a Square" << endl;
    }
};
```

Virtual Functions

```
int main(int argc, char *argv[])
{
    Shape* shape1 = new Circle();
    Shape* shape2 = new Square();

    shape1->draw();
    shape2->draw();

    delete shape1;
    delete shape2;

    return 0;
}
```

LibreOffice Writer

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ g++ vf02.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ ./a.out
Drawing a Circle
Drawing a Square
```

Google Chrome

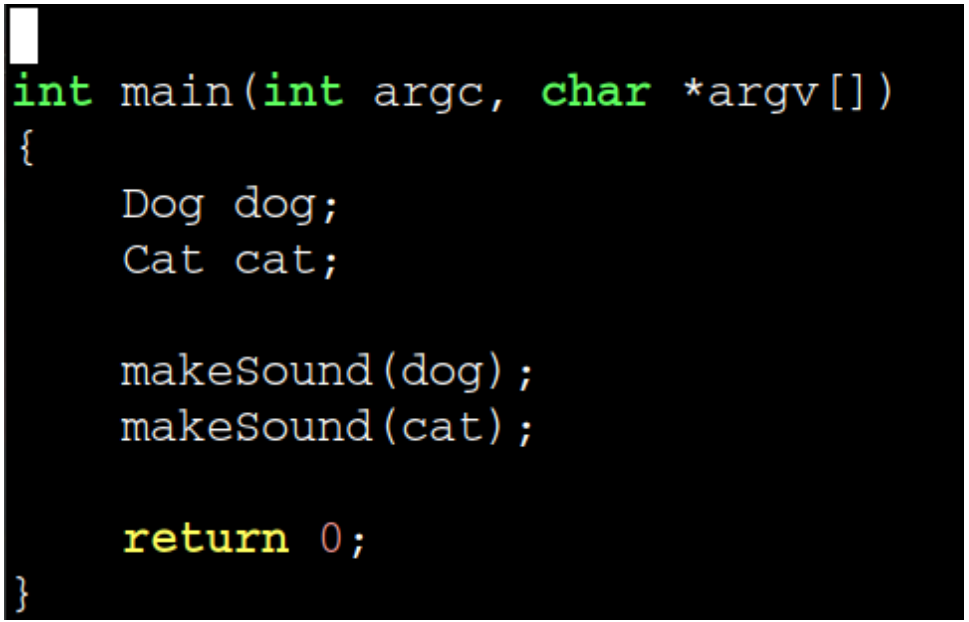
Virtual Functions & Inheritance Hierarchy

Virtual Functions & Inheritance Hierarchy

```cpp
#include <iostream>
using namespace std;

class Animal
{
public:
    virtual void sound() const
    {
        cout << "Some generic animal sound" << endl;
    }
};

class Dog : public Animal
{
public:
    void sound() const override
    {
        cout << "Woof!" << endl;
    }
};
```

```cpp
class Cat : public Animal
{
public:
    void sound() const override
    {
        cout << "Meow!" << endl;
    }
};

void makeSound(const Animal& animal)
{
    animal.sound();
}
```

Virtual Functions & Inheritance Hierarchy



```
int main(int argc, char *argv[])
{
    Dog dog;
    Cat cat;

    makeSound(dog);
    makeSound(cat);

    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ g++ vf03.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/07$ ./a.out
Woof!
Meow!
```

Templates

Templates

- Function templates
- Argument deduction
- Template parameters
- Overloading function templates
- Implementation of class templates
- Use of class templates
- Specialization of class templates
- Default template arguments

Function templates

Function Templates

- Function templates are functions
- Functions that are parameterized
- They represent a family of functions
- Examples
- This template definition specifies a family of functions that returns the maximum of two values

```
template <typename T>
T myMax(T a, T b)
{
    return a < b ? b : a;
}

int main(int argc, char *argv[])
{
    int a1 = 2, b1 = 5;
    char a2 = 'C', b2 = 'c';
    float a3 = 10.3, b3 = 8.3;
    double a4 = 20.75, b4 = 1.05;
    string a5 = "Hello", b5 = "CS204";

    cout << myMax(a1, b1) << endl;
    cout << myMax(a2, b2) << endl;
    cout << myMax(a3, b3) << endl;
    cout << myMax(a4, b4) << endl;
    cout << myMax(a5, b5) << endl;

    return 0;
}
```

Function Templates

- Following syntax precedes function name `template <typename T>`
- Once `typename` is specified, rest of the function uses this `typename` instead
- The typename is `T`
- Therefore, first input parameter is `T`
- Second input parameter is `T`
- Return type is `T`

```
template <typename T>
T myMax(T a, T b)
{
    return a < b ? b : a;
}

int main(int argc, char *argv[])
{
    int a1 = 2, b1 = 5;
    char a2 = 'C', b2 = 'c';
    float a3 = 10.3, b3 = 8.3;
    double a4 = 20.75, b4 = 1.05;
    string a5 = "Hello", b5 = "CS204";

    cout << myMax(a1, b1) << endl;
    cout << myMax(a2, b2) << endl;
    cout << myMax(a3, b3) << endl;
    cout << myMax(a4, b4) << endl;
    cout << myMax(a5, b5) << endl;

    return 0;
}
```

Function Templates

- When we call myMax, the template parameters are determined by the arguments passed.
- myMax(4, 7) OK
- myMax('C', 'c') OK
- Implicit type conversion is NOT allowed in templates
- myMax(4, 4.2) NOT OK

```
template <typename T>
T myMax(T a, T b)
{
    return a < b ? b : a;
}

int main(int argc, char *argv[])
{
    int a1 = 2, b1 = 5;
    char a2 = 'C', b2 = 'c';
    float a3 = 10.3, b3 = 8.3;
    double a4 = 20.75, b4 = 1.05;
    string a5 = "Hello", b5 = "CS204";

    cout << myMax(a1, b1) << endl;
    cout << myMax(a2, b2) << endl;
    cout << myMax(a3, b3) << endl;
    cout << myMax(a4, b4) << endl;
    cout << myMax(a5, b5) << endl;

    return 0;
}
```

Function Templates

- Three ways to have type casting
- myMax(static_cast<double>(4), 4.2)
- myMax<double>(4, 4.2)
- Specify that parameters may have different types.

```
template <typename T>
T myMax(T a, T b)
{
    return a < b ? b : a;
}

int main(int argc, char *argv[])
{
    int a1 = 2, b1 = 5;
    char a2 = 'C', b2 = 'c';
    float a3 = 10.3, b3 = 8.3;
    double a4 = 20.75, b4 = 1.05;
    string a5 = "Hello", b5 = "CS204";

    cout << myMax(a1, b1) << endl;
    cout << myMax(a2, b2) << endl;
    cout << myMax(a3, b3) << endl;
    cout << myMax(a4, b4) << endl;
    cout << myMax(a5, b5) << endl;

    return 0;
}
```

Function Templates

- Specify that parameters may have different types.
- Three parameter types
  - T1
  - T2
  - RT
- Return type RT
- Declaring `<typename T1, typename T2, typename RT>`
- Calling: `myMax<int, int, double>(a1, b1)`

```
#include<iostream>
using namespace std;

template <typename T1, typename T2, typename RT>
RT myMax(T1 a, T2 b)
{
    return a < b ? b : a;
}

int main(int argc, char *argv[])
{
    int a1 = 2, b1 = 5;
    char a2 = 'C', b2 = 'c';
    float a3 = 10.3, b3 = 8.3;
    double a4 = 20.75, b4 = 1.05;
    string a5 = "Hello", b5 = "CS204";

    cout << myMax<int, int, double> (a1, b1) << endl;
    cout << myMax<char, char, int>(a2, b2) << endl;
    cout << myMax<float, float, int>(a3, b3) << endl;
    cout << myMax<double, double, short int>(a4, b4) << endl;
    cout << myMax<string, string, string>(a5, b5) << endl;

    return 0;
}
```

Function Templates

- Specify that parameters may have different types.
- Three parameter types
  - RT
  - T1
  - T2
- Return type RT
- Declaring `<typename RT, typename T1, typename T2>`
- `Calling: myMax<double>(a1, b1)`

```
#include<iostream>
using namespace std;

template <typename RT, typename T1, typename T2>
RT myMax(T1 a, T2 b)
{
    return a < b ? b : a;
}

int main(int argc, char *argv[])
{
    int a1 = 2, b1 = 5;
    char a2 = 'C', b2 = 'c';
    float a3 = 10.3, b3 = 8.3;
    double a4 = 20.75, b4 = 1.05;
    string a5 = "Hello", b5 = "CS204";

    cout << myMax<double> (a1, b1) << endl;
    cout << myMax<int>(a2, b2) << endl;
    cout << myMax<int>(a3, b3) << endl;
    cout << myMax<short int>(a4, b4) << endl;
    cout << myMax<string>(a5, b5) << endl;

    return 0;
}
```

Overloading Function templates

Overloading Function Templates

- The myMax function is overloaded with
  - A non-template function
  - A template function with two arguments
  - A template function with three arguments
- These three functions can be invoked in multiple methods as shown in the example

Overloading Function Templates

```cpp
#include<iostream>
using namespace std;

int myMax(int a, int b)
{
  return a < b ? b: a;
}

template <typename T>
T myMax(T a, T b)
{
    return a < b ? b : a;
}

template <typename T>
T myMax(T a, T b, T c)
{
  return myMax(myMax(a, b), c);
}

int main(int argc, char *argv[])
{

    cout << myMax(7, 42, 68) << endl; //template with 3 arguments
    cout << myMax(7.0, 42.0) << endl; //myMax<double> (argument deduction)
    cout << myMax('a', 'b')  << endl; //myMax<char> (argument deduction)
    cout << myMax(7, 42)     << endl; //myMax<int> (argument deduction)
    cout << myMax<>(7, 42)   << endl; //myMax<int> (argument deduction)
    cout << myMax<double>(7, 42)   << endl; //myMax<double> (No argument deduction)
    cout << myMax('a', 42.7)   << endl; //non-template function with two arguments
    return 0;
}
```

Overloading Function Templates

- The myMax function is overloaded with
  - A template function with two arguments
  - A template function with pointers
  - A non-template function with argument as two pointer to char's

- These three functions can be invoked in multiple methods as shown in the example

Overloading Function Templates

```
#include<iostream>
using namespace std;
template <typename T> T myMax(T a, T b, T c)
{
  return myMax(myMax(a, b), c);
}

template <typename T> T* myMax(T *a, T *b)
{
  return *a < *b ? b : a;
}

char myMax(char *a, char *b)
{
  return strcmp(a, b) < 0 ? b: a;
}

int main(int argc, char *argv[])
{
    int a = 7, b = 42;
    cout << myMax(a, b) << endl; // two int values template
    string s1 = "Hey";
    string s2 = "You";
    cout << myMax(s1, s2) << endl; // two string values template

    int *p1 = &b;
    int *p2 = &a;
    cout << myMax(p1, p2) << endl; // two int* values template

    char *p3 = "David";
    char *p4 = "Nico";
    cout << myMax(p1, p2) << endl; // two int* values template

    return 0;
}
```

Class Templates

Class Templates

```cpp
#include <iostream>
using namespace std;

template <typename T>
class Box
{
  private:
    T content;

  public:
    Box(T c)
    {
      content = c;
    }

    void setContent(T c)
    {
        content = c;
    }

    T getContent() const
    {
        return content;
    }

    void display() const
    {
        cout << "The content of the box is: " << content << endl;
    }
};
```

```cpp
int main(int argc, char *argv[])
{
    Box<int> intBox(123);
    intBox.display();

    Box<double> doubleBox(45.67);
    doubleBox.display();

    Box<string> stringBox("Hello, World!");
    stringBox.display();

    stringBox.setContent("New Content");
    stringBox.display();

    return 0;
}
```

Default Class Template

Overloading Function Templates

- You have learned how to pass default values to functions
- Consider a constructor matrix(int r = 4, int c = 4)
- The object matrix m1; creates a matrix m1 of size 4x4
- The object matrix m2(4, 7); creates a matrix m2 of size 4x7.
- Similarly, one can create default types using templates.

Class Templates

```cpp
#include <iostream>
using namespace std;

template <typename T>
class Box
{
  private:
    T content;

  public:
    Box(T c)
    {
      content = c;
    }

    void setContent(T c)
    {
        content = c;
    }

    T getContent() const
    {
        return content;
    }

    void display() const
    {
        cout << "Box has: " << content << endl;
    }
};
```

```cpp
int main(int argc, char *argv[])
{
    Box<> intBox(123);
    intBox.display();

    Box<double> doubleBox(45.67);
    doubleBox.display();

    Box<string> stringBox("Hello, World!");
    stringBox.display();

    return 0;
}
```

Class Templates

```
hi@user:~/courses/cs204/july-dec-2024/tutorial/week07$ g++ template-06.cc -g
hi@user:~/courses/cs204/july-dec-2024/tutorial/week07$ ./a.out
ontent of the box is: 123
ontent of the box is: 45.67
ontent of the box is: Hello, World!
ontent of the box is: New Content
hi@user:~/courses/cs204/july-dec-2024/tutorial/week07$
```

Passing A Class As Type To Template Class

Passing Classes As Argument to Template Classes

Ratio class with two constructors and a print function

```
#include <iostream>
using namespace std;

class Ratio
{
private:
    int numerator;
    int denominator;

public:
    Ratio()
    {
      numerator = 10;
      denominator = 10;
    }
    Ratio(int num, int denom) : numerator(num), denominator(denom)
    {
        if (denom == 0) throw invalid_argument("Denominator cannot be zero.");
    }

    friend ostream& operator<<(ostream& os, const Ratio& ratio)
    {
        os << ratio.numerator << "/" << ratio.denominator;
        return os;
    }
};
```

Passing Classes As Argument to Template Classes

Polynomial class with one constructor and a print function

```
class Polynomial
{
private:
    int terms;
    int coeff[3];
    int degree[3];

public:
    Polynomial(int t = 3)
    {
      terms = t;
      int i = 0;
      for( i = 0; i < terms; i++ )
      {
        coeff[i] = degree[i] = 0;
      }
    }

    friend ostream& operator<<(ostream& os, const Polynomial& poly)
    {
        int i = 0;
        for (i = 0; i < poly.terms - 1; i++)
        {
          os << poly.coeff[i] << "x^" << poly.degree[i] << "+";
        }
        os << poly.coeff[i] << "x^" << poly.degree[i] << endl;
        return os;
    }
};
```

Passing Classes As Argument to Template Classes

Template Matrix Class

```
template <typename T>
class Matrix {
private:
    int rows, cols;
    T** data;

public:
  Matrix(int r, int c) : rows(r), cols(c)
  {
    int i = 0;
    data = new T*[rows];
    for (i = 0; i < rows; ++i)
    {
      data[i] = new T[cols];
    }
  }

  ~Matrix()
  {
    for (int i = 0; i < rows; ++i)
    {
      delete[] data[i];
    }
    delete[] data;
  }
```

```
  void setValue(int r, int c, const T& value)
  {
     data[r][c] = value;
  }

  T getValue(int r, int c) const
  {
      return data[r][c];
  }

  void display() const
  {
    int i = 0, j = 0;
    for(i = 0; i < rows; i++)
    {
      for(j = 0; j < cols; j++)
      {
        cout << data[i][j] << " ";
      }
      cout << endl;
    }
  }
};
```

Passing Classes As Argument to Template Classes

Template Matrix Class

```
int main(int argc, char *argv[])
{
    Matrix<Ratio> ratioMatrix(2, 2);
    ratioMatrix.setValue(0, 0, Ratio(1, 2));
    ratioMatrix.setValue(0, 1, Ratio(3, 7));
    ratioMatrix.setValue(1, 0, Ratio(2, 5));
    ratioMatrix.setValue(1, 1, Ratio(4, 9));

    cout << "Matrix of Ratios:" << endl;
    ratioMatrix.display();

    Matrix<Polynomial> polyMatrix(2, 2);
    polyMatrix.setValue(0, 0, Polynomial(3));
    polyMatrix.setValue(0, 1, Polynomial(3));
    polyMatrix.setValue(1, 0, Polynomial(3));
    polyMatrix.setValue(1, 1, Polynomial(3));

    cout << "Matrix of Polynomials:" << endl;
    polyMatrix.display();

    return 0;
}
```

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week07$ ./a.out
Matrix of Ratios:
1/2 3/7
2/5 4/9
Matrix of Polynomials:
0x^0+0x^0+0x^0
 0x^0+0x^0+0x^0

0x^0+0x^0+0x^0
 0x^0+0x^0+0x^0
```

Questions?

Thank You